Name: Arjun B R

Intern ID: IN126049802

Title: RAG-Based Customer Support Assistant

HIGH-LEVEL DESIGN (HLD)

1. Introduction

The Customer Support Assistant is a Retrieval-Augmented Generation (RAG) based system designed to answer user queries using information extracted from uploaded PDF documents. The system combines document processing, semantic search, and a language model to provide accurate responses.

Unlike traditional chatbots, this system does not rely only on pre-trained knowledge. Instead, it dynamically retrieves relevant information from user-provided documents and generates answers based on that context. Additionally, the system includes a Human-in-the-Loop (HITL) mechanism to handle cases where an answer cannot be generated confidently.

2. Objective

The objective of this High-Level Design is to define the overall system architecture of the RAG-based Customer Support Assistant. It outlines the major components, their interactions, and how data flows through the system to generate context-aware responses.

3. System Overview

Problem Definition

Traditional customer support systems often fail to provide accurate answers from large documents like policies, agreements, or manuals. They either rely on static responses or generate incorrect answers due to lack of context.

This project aims to solve this problem by building a system that can:

- Understand user queries
- Retrieve relevant information from documents
- Generate accurate, context-based answers

Scope of the System

The system allows users to:

- Upload a PDF document
- Ask questions related to that document
- Receive context-based answers
- If the system cannot find a suitable answer, it escalates to human input using a Human-in-the-Loop (HITL) mechanism.

The system is designed for:

- Customer support automation
- Document-based Q&A systems

4. Architecture Diagram

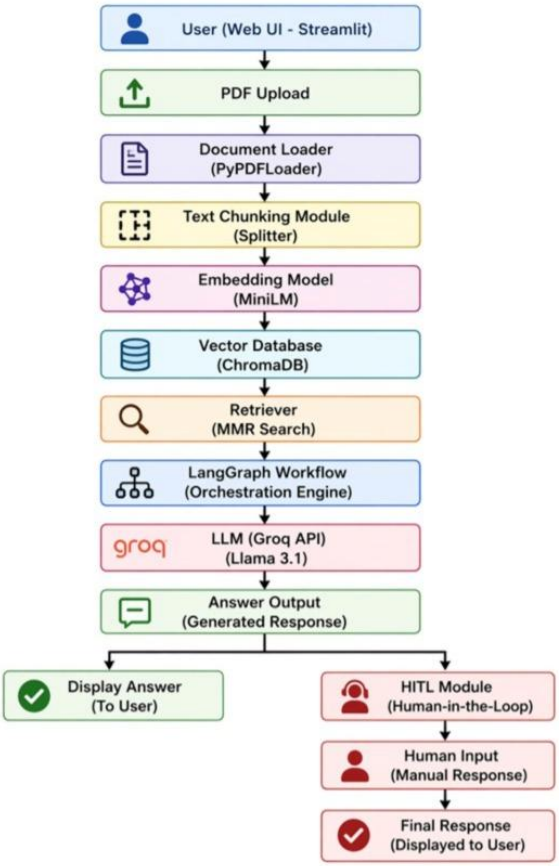

Fig: System Architecture Diagram

Components Covered:

- User Interface (Web - Streamlit)
- Document Ingestion Pipeline
- Embedding System
- Vector Database (ChromaDB)
- Retrieval Layer
- LLM Processing Layer (Groq API)
- Workflow Orchestration (LangGraph)
- HITL System

5. Component Description

- **Document Loader:**
  Loads the PDF and extracts text using PyPDFLoader.

- **Chunking Strategy:**
  Splits text into smaller chunks (700 characters with overlap) to improve retrieval accuracy.

- **Embedding Model:**
  Converts text chunks into vectors using MiniLM model for semantic understanding.

- **Vector Store:**
  Stores embeddings in ChromaDB for fast similarity search.

- **Retriever:**
  Uses Max Marginal Relevance (MMR) to fetch relevant and diverse chunks.

- **LLM:**
  Uses Groq API (LLaMA 3.1 model) to generate answers from retrieved context.

- **Graph Workflow Engine:**

  LangGraph manages the execution flow and decision-making logic.

- **Routing Layer**

  Determines whether to:

  - Return answer
  - Trigger HITL
  - HITL Module
  - Handles cases where no answer is found by requesting human input.

6. Data Flow

➢ **PDF to Answer Flow**

PDF → Text Extraction → Chunking → Embedding → ChromaDB → Retrieval → LLM → Answer

➢ **Query Lifecycle:**

- User submits a query
- System converts query into embedding
- Retriever fetches relevant chunks
- Context is sent to LLM
- LLM generates answer
- If no answer → HITL triggered
- Final response displayed

7. Technology Choices

❖ Why ChromaDB?

- Lightweight and fast
- Easy integration with LangChain
- Suitable for local vector storage

❖ **Why LangGraph?**

- Enables structured workflow
- Supports conditional routing
- Helps implement HITL easily

❖ **Why Groq (LLM)?**

- Fast inference speed
- Cost-effective
- Supports LLaMA models

❖ **Additional Tools**

- Streamlit → UI
- HuggingFace → embeddings
- Python → backend

8. Scalability Considerations

- **Handling Large Documents**

  The system uses a chunking strategy to divide large PDF documents into smaller, manageable text segments. This reduces memory usage and ensures that only relevant portions of the document are processed during retrieval. As a result, the system can efficiently handle documents of varying sizes without performance degradation.

- **Chunking Reduces Memory Load**

  Instead of processing the entire document at once, the system works with smaller chunks. This minimizes the computational load on the system and allows faster embedding generation and retrieval. It also improves accuracy by focusing on specific sections of the document rather than irrelevant content.

- **Vector Storage Enables Efficient Retrieval**

  All document chunks are converted into vector embeddings and stored in a vector database (ChromaDB). This allows the system to perform similarity-based search instead of keyword matching, making retrieval faster and more relevant even as the dataset grows.

- **Increasing Query Load**

  The use of vector indexing ensures that retrieval operations remain fast even when multiple queries are processed. Since embeddings are precomputed, the system only needs to perform similarity search and LLM inference, which helps in handling higher query volumes efficiently.

- **System Scalability Using Cloud Deployment**

  The system can be scaled by deploying components such as the vector database and LLM services on cloud platforms. This allows better handling of increased users, larger datasets, and higher traffic without affecting performance.

Latency Concerns

- Embeddings are computed once
- Retrieval is optimized using MMR
- LLM response time depends on APIHITL responses